



Creare, Leggere e Scrivere File Testuali con Python

Guida visiva alla gestione del File System.

Il Ciclo di Vita di un File

1. Aprire (Sbloccare)



`open()`

Informa il sistema operativo che vogliamo accedere a una risorsa.



2. Interagire



`read()` / `write()`

Il momento in cui manipoliamo i dati testuali nel file.



3. Chiudere (Sicurezza)



`close()`

Rilascia le risorse di sistema e salva definitivamente le modifiche.

Anatomia della funzione open()

Variabile File

Ospita l'oggetto file appena creato dal sistema.

Il Percorso

Indica dove si trova il file nel sistema.



Nota: Utilizziamo due slash (\) perché in Python il singolo slash (\) è un carattere di escape.

La Modalità (Mode)

Stabilisce le regole di ingaggio (es. **Scrittura** o Lettura).

```
file_uno = open("C:\\esempio_uno.txt", "w")
```

La Scrittura: write vs append

Modalità "w" (Write)



Azione Distruttiva

Se il file esiste, tutto il contenuto precedente viene **sovrascritto** e perso per sempre. Se non esiste, crea un **file nuovo**.

Modalità "a" (Append)



Azione Additiva

Il contenuto preesistente viene **conservato** in sicurezza. I nuovi dati si sommano in coda **alla fine** del documento.

Azione: Scrivere da zero (write)

```
contenuto = "Oggi è una bellissima giornata!"  
file_uno = open("esempio_uno.txt", "w")  
file_uno.write(contenuto)  
file_uno.close()
```



Oggi è una bellissima
giornata!

Ricorda: chiudere sempre il file
alla fine dell'operazione.



Azione: Aggiungere in coda (append)

```
nuova_stringa = "python è fantastico"  
file_uno = open("esempio_uno.txt", "a")  
file_uno.write(nuova_stringa)
```

Oggi è una bellissima
giornata!python è fantastico



**Attenzione: Nessun a
capo automatico**

Python non va a capo da solo. Usa il carattere
speciale *newline* per distanziare le stringhe.

```
file_uno.write("\nNuova_riga")
```


La Chiusura e il Paradigma Moderno

Perché `close()` è cruciale?



Salva definitivamente le modifiche sul disco.



Evita di raggiungere il limite massimo di file aperti dal sistema.



Interrompe l'uso delle risorse di memoria del computer.

L'Istruzione `with` (Il Paradigma Moderno)



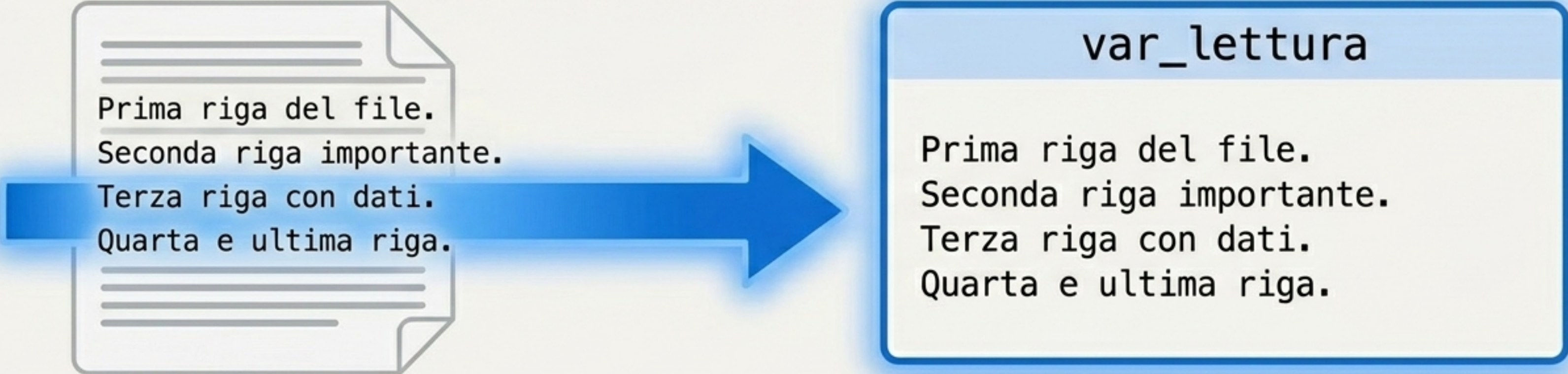
```
with open("file.txt", "r") as f:  
    f.read()
```



Invece di usare `close()` manualmente, il paradigma moderno usa l'istruzione `with`. Funziona come un pilota automatico: chiude il file in modo sicuro e garantito anche se si verificano errori nel codice.

La Lettura: Il parametro "r"

```
var_lettura = open("esempio_uno.txt", "r").read()  
print(var_lettura)
```



Prima riga del file.
Seconda riga importante.
Terza riga con dati.
Quarta e ultima riga.

var_lettura

Prima riga del file.
Seconda riga importante.
Terza riga con dati.
Quarta e ultima riga.

Il parametro "r" apre il file in sola lettura. Il metodo base `read()` importa tutto il contenuto in una singola stringa continua.

Matrice dei Metodi di Lettura

Best for Big Data		
<code>read()</code>	<code>readlines()</code>	<code>readline()</code>
		
Output: Una stringa gigante Importa l'intero file in una volta sola. Semplice da usare, ma rischioso per la memoria RAM se il file è molto pesante.	Output: Una Lista di stringhe <code>["Riga 1", "Riga 2", "\nNuova_riga"]</code> Legge tutte le righe del file e le salva come elementi separati all'interno di una lista Python. Usa molta memoria.	Output: Una stringa per volta Altamente efficiente. Estrae unicamente la riga corrente e prepara il cursore per estrarre la successiva. Il metodo migliore per file di grandi dimensioni.

Il Pattern per i Big Data: readline in azione

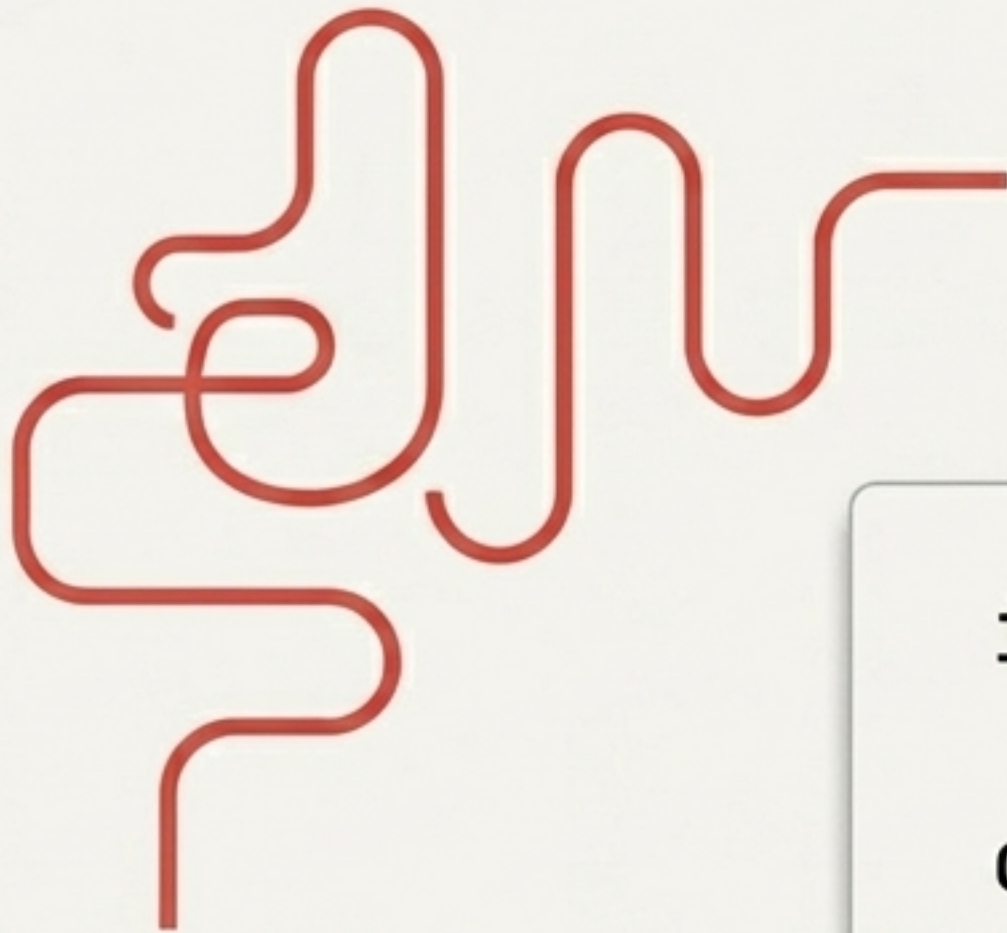
```
f = open("mio_file.txt", "r")
riga = f.readline()

while riga != "":
    print(riga)
    riga = f.readline()
```



L'efficienza al primo posto: carica in RAM solo ciò che stai leggendo in questo esatto millisecondo.

Orientarsi nel File System: Il modulo os



file.txt

Lavorare con molti file specificando ogni volta "C:\percorso\lungchissimo\file.txt" è inefficiente e soggetto a errori.

```
import os
```

```
os.chdir("/percorso/file")
```



Il modulo os funge da teletrasporto. Il comando `chdir` **cambia la cartella di lavoro di default** di Python. Ora si **può chiamare semplicemente** `open("file.txt")` **senza dover scrivere l'intero percorso assoluto.**

Cheat Sheet Finale

Modalità di Apertura

Modalità "w"

Scrive (Sovrascrive tutto dal principio)

Modalità "a"

Aggiunge (Conserva e inserisce in coda)

Modalità "r"

Legge (Solo lettura, estrae i dati)

I Metodi di Azione

`write()`

Inserisce il testo.
Ricorda: usa `\n` per forzare i rientri.

`read()`

Estrae in blocco in una singola, grande Stringa.

`readlines()`

Estrae il documento in una Lista Python `[]`.

`readline()`

Estrae riga per riga. Ideale per Loop e Big Data.

La Regola d'Oro



Usa SEMPRE `close()` o il costrutto `with` per chiudere il file, rilasciare la memoria e blindare i tuoi dati.

